

Legal Document Analyzer

Prepared By: Bhargav Doddi

Role: AI Automation Engineer

Project Category: RAG (Retrieval-Augmented Generation) System

Assessment Framework: GenAI Assessment Framework 2026

Date: July 2026

Document Information

Item	Details
Project Name	Legal Document Analyzer
Document Type	Software Design Document
Version	1.0
Status	Final
Prepared By	Bhargav Doddi
Programming Language	Python 3.12+
Frontend	React.js

Backend	FastAPI
AI Framework	LangChain
Large Language Model	Groq Llama 3.3 70B (llama-3.3-70b-versatile)
Vector Database	ChromaDB
Database	SQLite
Deployment	Vercel (Frontend), Railway (Backend)

Revision History

Version	Date	Description	Author
1.0	July 2026	Initial Software Design Document	Bhargav Doddi

Executive Summary

Legal documents such as contracts, agreements, policies, and regulations often contain lengthy and complex information that is difficult to understand and analyse manually. Searching for specific clauses or extracting relevant information from these documents can be time-consuming and error-prone.

The **Legal Document Analyzer** is an AI-powered Retrieval-Augmented Generation (RAG) application designed to simplify legal document analysis. Users can upload legal documents in PDF and DOCX formats, and ask questions in natural language. Instead of relying solely on the Large Language Model, the system retrieves relevant document sections from a vector database and combines them with **Groq Llama 3.3 70B (llama-3.3-70b-versatile)** to generate accurate, context-aware responses.

The application is developed using **React.js** for the frontend, **FastAPI** for backend services, **LangChain** for orchestrating the RAG pipeline, **Groq Llama 3.3 70B (llama-3.3-70b-versatile)** for response generation, **ChromaDB** for vector storage, and **SQLite** for managing document metadata. This architecture enables efficient document retrieval while reducing AI hallucinations and improving response accuracy.

This Software Design Document describes the system architecture, database design, API specification, user interface, testing strategy, deployment process, and future enhancements of the Legal Document Analyzer. It serves as the primary technical reference for the development and maintenance of the application.

Table of Contents

1. Introduction
2. Overall System Description
3. Software Architecture
4. Technology Stack
5. Database Design
6. API Design
7. User Interface Design
8. Testing Strategy
9. Deployment and Security
10. Future Enhancements and Conclusion
11. References

Chapter 1 – Introduction

1.1 Overview

Legal professionals, organisations, and individuals frequently work with lengthy legal documents containing contracts, agreements, policies, and regulatory information. Manually reviewing these documents requires significant time and effort, particularly when searching for specific clauses or answering legal questions.

The **Legal Document Analyzer** is an AI-powered application that simplifies this process by allowing users to upload legal documents and interact with them using natural language queries. The application applies Retrieval-Augmented Generation (RAG) to retrieve the most relevant document content before generating responses with Groq. This approach improves response accuracy while ensuring answers remain grounded in the uploaded documents.

1.2 Objectives

The primary objectives of the Legal Document Analyzer are:

- Analyse legal documents efficiently.
- Answer user queries using document context.
- Reduce manual document review time.
- Retrieve relevant legal information accurately.
- Provide context-aware AI-generated responses.
- Store uploaded document metadata for future reference.

1.3 Scope

The application supports uploading PDF and Docx legal documents, extracting text, generating embeddings, storing document vectors, and answering user questions using Retrieval-Augmented Generation. The system is designed for document analysis and information retrieval rather than providing legal advice or making legal decisions.

1.4 Target Users

User Category	Purpose
Lawyers	Review legal documents efficiently
Legal Consultants	Analyse contracts and agreements
Businesses	Search company policies and legal records
Students	Study legal documents and regulations

1.5 Expected Benefits

- Faster legal document analysis.
- Accurate context-based question answering.
- Reduced manual effort in document review.
- Improved accessibility of legal information.
- Scalable architecture for handling multiple documents.

1.6 Assumptions

- Users upload readable PDF and Docx documents.
- Groq API is available.
- ChromaDB stores document embeddings successfully.
- Internet connectivity is available during document processing.

1.7 Constraints

- Supports PDF and Docx documents in the current version.
- Response quality depends on uploaded document content.
- Internet connection is required for AI processing.
- The application provides informational assistance only and does not replace professional legal advice.

Chapter 2 – Overall System Description

2.1 Overview

The Legal Document Analyzer is a Retrieval-Augmented Generation (RAG) application designed to help users understand and analyse legal documents efficiently. Instead of manually searching through lengthy contracts, agreements, or policies, users can upload documents and ask questions in natural language. The system retrieves the most relevant document sections using semantic search and generates accurate responses using **Groq Llama 3.3 70B (llama-3.3-70b-versatile)**.

The application follows a modular client-server architecture where the React frontend communicates with the FastAPI backend. The backend manages document processing, embedding generation, vector storage, AI integration, and response generation.

2.2 Existing System

Traditional legal document review is largely performed manually. Users typically read lengthy PDF documents or search using simple keywords, making it difficult to quickly locate relevant clauses or understand complex legal language.

Limitations of Existing Systems

- Manual review is time-consuming.
- Keyword searches often miss relevant context.
- Difficult to analyse lengthy legal documents.
- Limited support for natural language queries.

- No intelligent document understanding.
- Increased possibility of human error.

2.3 Proposed System

The proposed Legal Document Analyzer combines Retrieval-Augmented Generation (RAG) with Large Language Models to improve document understanding. After a user uploads a legal document, the system extracts text, divides it into manageable chunks, generates vector embeddings, and stores them in ChromaDB. When a question is asked, the system retrieves the most relevant document sections and sends them to Groq Llama 3.3 70B (llama-3.3-70b-versatile) to generate an accurate, context-aware response.

2.4 System Components

Component	Description
User Interface	Allows users to upload documents and ask questions.
Document Processor	Extracts text from uploaded PDF and DOCX files
Text Chunking Module	Divides extracted text into smaller sections.
Embedding Generator	Converts text chunks into vector embeddings.
Vector Database	Stores embeddings for semantic retrieval.
AI Engine	Generates responses using retrieved document context.
Metadata Database	Stores document information and processing history.

2.5 Functional Requirements

ID	Requirement
FR-01	Uploads and validates PDF and DOCX documents
FR-02	Extract text from uploaded documents.
FR-03	Generate embeddings for document chunks.
FR-04	Store embeddings in ChromaDB.
FR-05	Accept natural language questions from users.
FR-06	Retrieve relevant document content.
FR-07	Generate AI-powered answers using Groq
FR-08	Store document metadata and processing history.

2.6 Non-Functional Requirements

Category	Requirement
Performance	Process uploaded documents efficiently.
Reliability	Ensure accurate document retrieval.

Security	Protect uploaded documents and API credentials.
Scalability	Support future migration to cloud databases.
Availability	Maintain consistent application availability.
Maintainability	Modular architecture for easy updates.
Usability	Simple and intuitive user interface.

2.7 User Roles

User	Responsibilities
User	Upload documents, ask questions, and view AI-generated responses.
Administrator	Manage system configuration, monitor services, and maintain application performance.

2.8 System Workflow

Figure 2.1: Overall System Workflow

Upload PDF/Docx → Extract Text → Chunk Text → Generate Embeddings → Store in ChromaDB → User Query → Retrieve Relevant Chunks → Groq Llama 3.3 70B (llama-3.3-70b-versatile) → AI Response

2.9 Assumptions

- Users upload valid PDF documents.
- Groq API remains available.
- ChromaDB stores embeddings successfully.
- Uploaded documents contain readable text.

- Internet connectivity is available during processing.

2.10 Constraints

- Only PDF documents are supported in the current version.
- AI responses depend on the uploaded document content.
- Processing large documents may increase response time.
- The application is intended for document analysis and should not be considered a substitute for professional legal advice.

Chapter 3 – Software Architecture

3.1 Overview

The Legal Document Analyzer follows a modular client-server architecture that separates the presentation layer, application logic, Artificial Intelligence processing, and data storage. This architecture improves maintainability, scalability, and simplifies future enhancements.

The React frontend provides the user interface for uploading documents and submitting queries. FastAPI manages business logic and coordinates communication with the document processing module, ChromaDB, SQLite, and Groq.

3.2 Architecture Style

The application adopts a **Client–Server Architecture** combined with a **Retrieval-Augmented Generation (RAG)** pipeline.

- React.js provides the web interface.
- FastAPI processes client requests.
- LangChain manages the RAG workflow.
- ChromaDB performs semantic document retrieval.
- Groq generates context-aware responses.
- SQLite stores document metadata and application records.

3.3 Overall Architecture

Figure 3.1: Overall System Architecture

User → React Frontend → FastAPI Backend → Document Processing → Embedding Generation → ChromaDB → Groq → AI Response

3.4 Architecture Layers

Layer	Responsibility
Presentation Layer	User interface for document upload and question answering.
Application Layer	Handles business logic, request processing, and workflow management.
AI Processing Layer	Performs document retrieval and AI response generation.
Data Layer	Stores document metadata and vector embeddings.

3.5 System Modules

Module	Description
Authentication Module	manages user registration, login, and JWT-based authorization.
Document Processing Module	Uploads and validates PDF and Docx documents.
PDF Processing Module	Extracts text from uploaded documents.
Chunking Module	Splits extracted text into smaller segments.
Embedding Module	Generates vector embeddings for document chunks.
Vector Search Module	Retrieves relevant document sections using semantic search.

AI Response Module	Generates answers using Groq.
Metadata Module	Stores document details and processing history in SQLite.

3.6 Module Interaction

The workflow begins when a user uploads a legal document through the React application. The backend validates the file and extracts its text using the document processing module. The extracted content is divided into smaller chunks, converted into vector embeddings, and stored in ChromaDB.

When the user submits a question, the backend performs semantic similarity search to retrieve the most relevant document chunks. These retrieved sections are combined with the user's query and sent to Groq . The generated response is then returned to the frontend for display.

3.7 RAG Workflow

Figure 3.2: Retrieval-Augmented Generation Workflow

Upload Document → Extract Text → Create Chunks → Generate Embeddings
→ Store in ChromaDB → User Query → Retrieve Relevant Chunks → Generate
Answer → Display Response

3.8 Design Principles

Principle	Description
Modularity	Independent modules simplify maintenance and upgrades.
Scalability	Components can be extended without affecting the overall architecture.
Reusability	Services can be reused across different workflows.

Separation of Concerns	Each module performs a single responsibility.
Reliability	Semantic retrieval improves answer accuracy and reduces hallucinations.

3.9 Architecture Benefits

- Modular and maintainable application structure.
- Efficient document retrieval using vector search.
- Improved response accuracy through Retrieval-Augmented Generation.
- Easy integration of additional document formats in future versions.
- Supports migration to cloud-based infrastructure.
- Simplifies future enhancements and feature expansion.

Chapter 4 – Technology Stack

4.1 Overview

The Legal Document Analyzer is developed using modern web technologies and Artificial Intelligence frameworks that support Retrieval-Augmented Generation (RAG). The selected technologies enable efficient document processing, semantic search, AI-powered response generation, and reliable application performance.

4.2 Technology Stack

Component	Technology
Frontend	React.js + Tailwind CSS
Backend	FastAPI
Programming Language	Python 3.12+

AI Framework	LangChain
Large Language Model	Groq Llama 3.3 70B Ve
Document Processing	PyMuPDF + python-docx
Embeddings	sentence-transformers/a
Vector Database	ChromaDB
Database	SQLite
Version Control	Git & GitHub
Deployment	Vercel & Railway

4.3 System Workflow

Figure 4.1: Technology Workflow

React → FastAPI → LangChain → ChromaDB → Groq → Response

4.4 Development Tools

Tool	Purpose
Visual Studio Code	Application development
Git & GitHub	Version control

Swagger	API testing
Chrome	Browser testing

4.5 Technology Benefits

- Lightweight and modular architecture.
- Efficient REST API communication.
- Accurate semantic document retrieval.
- AI-powered contextual responses.
- Simple deployment and maintenance.
- Scalable for future enhancements.

Chapter 5 – Database Design

5.1 Overview

The Legal Document Analyzer uses a hybrid storage architecture consisting of **SQLite** and **ChromaDB**. SQLite stores structured application data such as document information and query history, while ChromaDB stores vector embeddings generated from document text. This combination enables efficient document management and fast semantic retrieval for Retrieval-Augmented Generation (RAG).

5.2 Database Components

Database	Purpose
SQLite	Stores document metadata and query history
ChromaDB	Stores vector embeddings for semantic search

5.3 Document Metadata

Field	Description
Document ID	Unique document identifier
Document Name	Uploaded PDF file name
Upload Date	Date and time of upload
File Type	Document format
Processing Status	Current processing status

5.4 Query History

Field	Description
Query ID	Unique query identifier
Document ID	Reference to uploaded document
User Question	User's search query
AI Response	Generated answer
Timestamp	Query execution time

5.5 Database Workflow

Figure 5.1: Database Workflow

Upload Document → Store Metadata → Generate Embeddings → Save in ChromaDB → Retrieve Relevant Chunks → Generate AI Response

5.6 CRUD Operations

Operation	Purpose
Create	Save uploaded documents and metadata
Read	Retrieve document information and vectors
Update	Modify processing status
Delete	Remove documents and related records

5.7 Database Benefits

- Hybrid storage architecture for structured and vector data.
- Fast semantic document retrieval.
- Efficient management of uploaded documents.
- Supports Retrieval-Augmented Generation (RAG).
- Easy to maintain and extend for future cloud databases.

Chapter 6 – API Design

6.1 Overview

The Legal Document Analyzer uses RESTful APIs to enable communication between the React frontend and the FastAPI backend. The APIs manage document uploads, document processing, semantic search, and AI-powered question answering. Each endpoint is designed to provide reliable and efficient interaction while maintaining a simple and scalable architecture.

6.2 API Endpoints

Method
POST
POST
POST
GET
DELETE
POST
GET

6.3 API Workflow

Figure 6.1: API Communication Flow

User → React Frontend → FastAPI → ChromaDB → Groq → AI Response

6.4 HTTP Status Codes

Code	Description
200	Request completed successfully

201	Document uploaded successfully
400	Invalid request or unsupported file
404	Document not found
500	Internal server error

6.5 API Security

Security Feature	Purpose
Input Validation	Validates uploaded files and user input
HTTPS	Secures communication between client and server
Environment Variables	Protects API credentials
Error Handling	Prevents exposure of sensitive information

6.6 API Features

- Simple RESTful architecture.
- Supports secure document upload.
- Enables semantic document retrieval.
- Generates context-aware AI responses.
- Modular endpoints for easy maintenance and future expansion.

Chapter 7 – User Interface Design

7.1 Overview

The Legal Document Analyzer provides a clean and intuitive web interface that enables users to upload legal documents, ask questions in natural language, and view AI-generated responses. The interface is designed to minimise user effort while ensuring smooth navigation and an efficient document analysis experience.

7.2 User Interface Screens

Screen	Description
Home Page	Displays application overview and navigation options.
Document Upload	Allows users to upload PDF legal documents.
Document List	Displays uploaded documents available for analysis.
Question & Answer	Enables users to ask questions and view AI responses.
Response History	Displays previous questions and generated answers.

7.3 Navigation Flow

Figure 7.1: User Interface Workflow

Home → Upload Document → Process Document → Ask Question → View AI Response → Response History

7.4 User Interface Components

Component	Purpose
Navigation Bar	Access different application sections.
Upload Button	Select and upload PDF documents.

Search Box	Enter legal questions in natural language.
Response Panel	Display AI-generated answers.
History Panel	View previous queries and responses.

7.5 Design Principles

Principle	Description
Simplicity	Clean and easy-to-use interface.
Consistency	Uniform layout across all pages.
Responsiveness	Supports desktop and mobile devices.
Accessibility	Easy navigation and readable content.
User Experience	Quick access to document analysis features.

7.6 User Interface Benefits

- Simple and intuitive navigation.
- Quick document upload and processing.
- Easy interaction using natural language questions.
- Clear presentation of AI-generated responses.
- Responsive design for multiple devices.

Chapter 8 – Testing Strategy

8.1 Overview

Testing is an essential phase of the software development lifecycle that ensures the Legal Document Analyzer performs reliably and meets its functional requirements. The application is tested to verify document upload, text extraction, embedding generation, semantic retrieval, AI response generation, and overall system performance.

8.2 Testing Objectives

The primary objectives of testing are:

- Verify all functional requirements.
- Ensure accurate document processing.
- Validate semantic search results.
- Test AI-generated responses.
- Identify and resolve software defects.
- Improve system reliability and performance.

8.3 Testing Types

Testing Type	Purpose
Unit Testing	Tests individual modules independently.
Integration Testing	Verifies communication between system components.
Functional Testing	Validates application features against requirements.
System Testing	Evaluates complete application functionality.
User Acceptance Testing	Confirms the system satisfies user expectations.

8.4 Test Scenarios

Test ID	Scenario	Expected Result
TS-01	Upload PDF/Docx document	Document uploaded successfully
TS-02	Process uploaded document	Text extracted and embeddings generated
TS-03	Ask a legal question	AI returns a context-based response
TS-04	Retrieve document information	Correct document details displayed
TS-05	Upload unsupported file	Appropriate validation error displayed

8.5 Testing Tools

Tool	Purpose
Swagger	API testing
Pytest	Backend unit testing
React Testing Library	Frontend component testing
Google Chrome	User interface testing
GitHub	Version control and test management

8.6 Test Execution Process

Figure 8.1: Testing Workflow

Unit Testing → Integration Testing → Functional Testing → System Testing →
User Acceptance Testing → Production Deployment

8.7 Expected Outcome

Successful testing ensures that the Legal Document Analyzer processes legal documents correctly, retrieves relevant information accurately, generates reliable AI-powered responses, and delivers a secure and user-friendly experience.

Chapter 9 – Deployment and Security

9.1 Overview

The Legal Document Analyzer is deployed as a web-based application using a client-server architecture. The React frontend communicates with the FastAPI backend through REST APIs, while the backend integrates with Groq for AI-powered responses and ChromaDB for semantic document retrieval. The deployment architecture is designed to provide secure, reliable, and scalable access to the application.

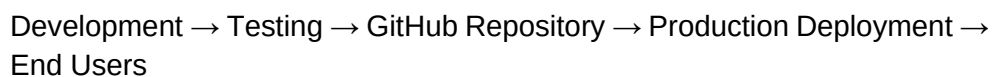
9.2 Deployment Environment

Component	Technology
Frontend	React.js
Backend	FastAPI
AI Service	Groq API
Vector Database	ChromaDB

Relational Database	SQLite
Deployment Platform	Vercel (Frontend), Railway (Backend)
API Communication	HTTPS REST API

9.3 Deployment Workflow

Figure 9.1: Deployment Process



9.4 Security Features

Security Measure	Purpose
HTTPS	Encrypts communication between client and server.
File Validation	Accepts only supported document formats.
Input Validation	Prevents invalid or malicious user input.
Environment Variables	Secures API keys and configuration settings.
Error Handling	Prevents exposure of sensitive system information.

9.5 Deployment Steps

1. Develop and test the application locally.

2. Push the source code to the GitHub repository.
3. Configure environment variables.
4. Deploy the backend on Railway.
5. Deploy the frontend on Vercel.
6. Verify application functionality after deployment.

9.6 Deployment Benefits

- Secure communication using HTTPS.
- Reliable cloud-based deployment.
- Easy application maintenance and updates.
- Scalable architecture for future growth.
- Supports continuous development and deployment.

9.7 Future Deployment Scope

Future versions of the application can be deployed using Docker containers and cloud platforms such as AWS, Microsoft Azure, or Google Cloud Platform. SQLite may also be replaced with PostgreSQL to improve scalability and support larger document collections.

Chapter 10 – Conclusion and Future Enhancements

10.1 Conclusion

The Legal Document Analyzer is an AI-powered web application that simplifies the analysis of legal documents by combining Retrieval-Augmented Generation (RAG) with modern web technologies. The system enables users to upload legal documents, perform semantic searches, and receive context-aware answers generated by Groq.

The application follows a modular architecture using React.js, FastAPI, ChromaDB, and SQLite, making it scalable, maintainable, and efficient. By integrating vector search with large language models, the system improves information retrieval and reduces the time required to analyse lengthy legal documents.

Overall, the proposed solution provides a secure, user-friendly, and reliable platform for legal document analysis while serving as a strong foundation for future AI-driven legal applications.

10.2 Future Enhancements

The following improvements can further enhance the capabilities of the Legal Document Analyzer:

- Support additional document formats such as TXT.
- Enable multi-document comparison and analysis.
- Integrate cloud databases such as PostgreSQL for improved scalability.
- Add multilingual document processing and translation support.
- Develop advanced legal analytics and document summarisation features.
- Deploy the application using Docker and Kubernetes for easier scalability and maintenance.
- Introduce real-time collaboration and document sharing capabilities.

References

The following references were used during the design and development of the **Legal Document Analyzer** project and while preparing this Software Design Document.

1. LangChain. (2026). *LangChain Documentation*. Available at: <https://python.langchain.com/>
2. Groq. (2026). Groq API Documentation <https://console.groq.com/docs>
3. Chroma. (2026). *ChromaDB Documentation*. Available at: <https://docs.trychroma.com/>
4. FastAPI. (2026). *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com/>
5. React. (2026). *React Documentation*. Available at: <https://react.dev/>
6. Tailwind CSS. (2026). *Tailwind CSS Documentation*. Available at: <https://tailwindcss.com/docs>
7. SQLite. (2026). *SQLite Documentation*. Available at: <https://www.sqlite.org/docs.html>
8. PyMuPDF. (2026). *PyMuPDF Documentation*. Available at: <https://pymupdf.readthedocs.io/>
9. Python Software Foundation. (2026). *Python Documentation*. Available at: <https://docs.python.org/3/>
10. OpenAPI Initiative. (2026). *OpenAPI Specification*. Available at: <https://swagger.io/specification/>
11. IEEE. (2021). *IEEE Std 1016-2021 – IEEE Standard for Software Design Description*. IEEE Standards Association.
12. Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education.
13. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill Education.
14. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
15. GenAI Assessment Framework 2026. *Project Guidelines and Assessment Requirements*. Internal Assessment Document.